SMART METER ANOMALY DETECTION

Using an LSTM Autoencoder on Hourly Electricity Data

Deep Learning Project Report

Abstract:

This project focuses on anomaly detection in household electricity consumption using deep learning. The UCI Household Electric Power Consumption dataset was converted into hourly averages and missing values were filled by interpolation. An LSTM Autoencoder model was trained on 24-hour windows of electricity consumption to spot anomalies by looking at reconstruction error. For testing 60 artificial anomalies – 30 spikes and 30 sudden drops – were added to the data. The LSTM Autoencoder model was compared with Z-score and Isolation Forest methods and its performance was measured by Precision, Recall and F1-score. The LSTM Autoencoder model achieved the F1-score of 0.640 with 0.853 Precision and 0.512 Recall. An interactive Streamlit dashboard lets users change detection thresholds, explore the data, and upload their files. The main limitations of this study are the use of anomalies and data from only one household.

Introduction:

A smart meter records household electricity consumption at intervals. Unusual readings, called anomalies may show appliances, meter problems, electricity theft or power cuts. Checking records by hand is hard so an automatic anomaly detection system is needed. This project uses an LSTM Autoencoder model to learn electricity consumption patterns and spot anomalies. The LSTM Autoencoder model is compared with Z-score and Isolation Forest methods and its success is measured by Precision, Recall and F1-score. The system also gives a Streamlit dashboard.

CODE:

```
# ====Smart Meter Anomaly Detection: Core Pipeline ====
import numpy as np, pandas as pd, joblib
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import precision_score, recall_score, f1_score
from tensorflow import keras
from tensorflow.keras import layers

WINDOW = 24

# 1. Preprocessing (preprocess.py)
df = pd.read_csv("data/household_power_consumption.txt",
                 sep=";", na_values="?", low_memory=False)

df["datetime"] = pd.to_datetime(df["Date"] + " " + df["Time"],
                                format="%d/%m/%Y %H:%M:%S")

series = df.set_index("datetime")["Global_active_power"].astype(float)
series = series.resample("h").mean()
series = series.interpolate(limit_direction="both")
```

```
split = int(len(series) * 0.8)
train_raw, test_raw = series.iloc[:split], series.iloc[split:]

scaler = MinMaxScaler()
train_scaled = scaler.fit_transform(train_raw.values.reshape(-1, 1))
test_scaled = scaler.transform(test_raw.values.reshape(-1, 1))

def make_windows(data, window):
    return np.array([data[i : i + window] for i in range(len(data) - window + 1)])

X_train = make_windows(train_scaled, WINDOW)
X_test = make_windows(test_scaled, WINDOW)
```

#2. LSTM Autoencoder (train.py)

```
inputs = keras.Input(shape=(WINDOW, 1))
x = layers.LSTM(32, return_sequences=True)(inputs)
x = layers.LSTM(16)(x)
x = layers.RepeatVector(WINDOW)(x)
x = layers.LSTM(16, return_sequences=True)(x)
x = layers.LSTM(32, return_sequences=True)(x)
outputs = layers.TimeDistributed(layers.Dense(1))(x)

model = keras.Model(inputs, outputs)
model.compile(optimizer="adam", loss="mae")

early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=5,
                                           restore_best_weights=True)
model.fit(X_train, X_train, epochs=30, batch_size=64,
          validation_split=0.1, callbacks=[early_stop])
```

3. Anomaly score and threshold (train.py / recalibrate.py)

```
def max_error(X):
    return np.max(np.abs(model.predict(X, verbose=0) - X), axis=(1, 2))

train_err, test_err = max_error(X_train), max_error(X_test)
threshold = np.percentile(train_err, 97)
anomalies = test_err > threshold

model.save("data/lstm_ae.keras")
joblib.dump(scaler, "data/scaler.pkl")
np.save("data/threshold.npy", threshold)
np.save("data/test_err.npy", test_err)
```

4. Evaluation with injected anomalies (evaluate.py)

```
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(42)
test_vals = test_raw.values.astype(float)
n = len(test_vals)

idx = rng.choice(np.arange(WINDOW, n - WINDOW), size=60, replace=False)
corrupted = test_vals.copy()
corrupted[idx[:30]] += rng.uniform(3, 6, size=30)       # 30 spikes
corrupted[idx[30:]] = 0.0                               # 30 sudden drops

is_anomaly = np.zeros(n, dtype=bool)
is_anomaly[idx] = True
labels = np.array([is_anomaly[i : i + WINDOW].any() for i in range(n - WINDOW + 1)])

X_corr = make_windows(scaler.transform(corrupted.reshape(-1, 1)), WINDOW)

# Baseline 1: Z-score (largest z-score in each window)
mu, sd = X_train[:, :, 0].mean(), X_train[:, :, 0].std()
z_tr = (np.abs(X_train[:, :, 0] - mu) / sd).max(axis=1)
z_te = (np.abs(X_corr[:, :, 0] - mu) / sd).max(axis=1)

# Baseline 2: Isolation Forest (on flattened 24-hour windows)
iso = IsolationForest(n_estimators=100, random_state=42)
iso.fit(X_train.reshape(len(X_train), -1))
iso_tr = -iso.score_samples(X_train.reshape(len(X_train), -1))
iso_te = -iso.score_samples(X_corr.reshape(len(X_corr), -1))

# Compare all methods with the same 95th percentile threshold
for name, tr, te in [("Z-score", z_tr, z_te),
                     ("Isolation Forest", iso_tr, iso_te),
                     ("LSTM AE (max error)", train_err, max_error(X_corr))]:
    flags = te > np.percentile(tr, 95)
    print(f"{name:20} P={precision_score(labels, flags):.3f} "
          f"R={recall_score(labels, flags):.3f} F1={f1_score(labels, flags):.3f}")
```

#5. Dashboard logic (app.py, Streamlit)

```
import streamlit as st
import matplotlib.pyplot as plt
base_threshold = float(np.load("data/threshold.npy"))
factor = st.sidebar.slider("Threshold multiplier (higher = fewer anomalies)", 0.5, 2.0, 1.0, 0.1)
thr = base_threshold * factor

test_err = np.load("data/test_err.npy")
```

```
y = np.load("data/processed.npz")["test_raw"][WINDOW - 1:]
days = st.slider("Days to show", 7, 90, 30)
flags = test_err[: days * 24] > thr

fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(y[: days * 24], color="steelblue", label="Consumption (kW)")
ax.scatter(np.where(flags)[0], y[: days * 24][flags], color="red", label="Anomaly")
st.pyplot(fig)
st.metric("Anomalies found", int(flags.sum()))
```

Methodology:

The UCI Household Electric Power Consumption dataset was used, with the Global_active_power variable (in kilowatts). The per-minute readings were converted to averages (34,589 hours). Missing values were filled by interpolation. The data was split chronologically into 80% training and 20% testing. Then it was scaled to the 0–1 range using a MinMax scaler fitted on the training data only. The data was divided into sliding windows of 24 hours. An LSTM Autoencoder was trained to reconstruct windows. Each window was scored by its reconstruction error. Both the mean error and the maximum error were tested as anomaly scores. In the system windows whose maximum error is above the 97th percentile of the training errors are flagged as anomalies. For evaluation 60 artificial anomalies (30 spikes and 30 drops, to zero) were injected into the test data. The model was compared with Z-score and Isolation Forest baselines using Precision, Recall and F1-score. All methods were compared using the 95th percentile threshold. The system is delivered as a Streamlit dashboard in which users can adjust the threshold and upload their data.

Result:

Table 1: Performance comparison of the methods

Method	Precision	Recall	F1-score	Windows flagged
Z-score	0.909	0.358	0.514	528
Isolation Forest	0.122	0.004	0.007	41
LSTM AE (mean error)	0.804	0.267	0.401	445
LSTM AE (max error)	0.853	**0.512**	**0.640**	804

comparison:

The injected anomalies affected 1,339 of the 6,895 test windows, since each anomalous hour appears in up to 24 overlapping windows. Table 1 compares the methods at the same 95th percentile threshold. The LSTM Autoencoder with maximum reconstruction error achieved the best F1-score (0.640), with a Precision of 0.853 and the highest Recall (0.512).

The Z-score method reached a higher Precision (0.909) but a lower Recall (0.358), so it missed more anomalies (F1 = 0.514). Isolation Forest performed poorly (F1 = 0.007), and the LSTM Autoencoder using mean error was weaker than the max-error version (F1 = 0.401).

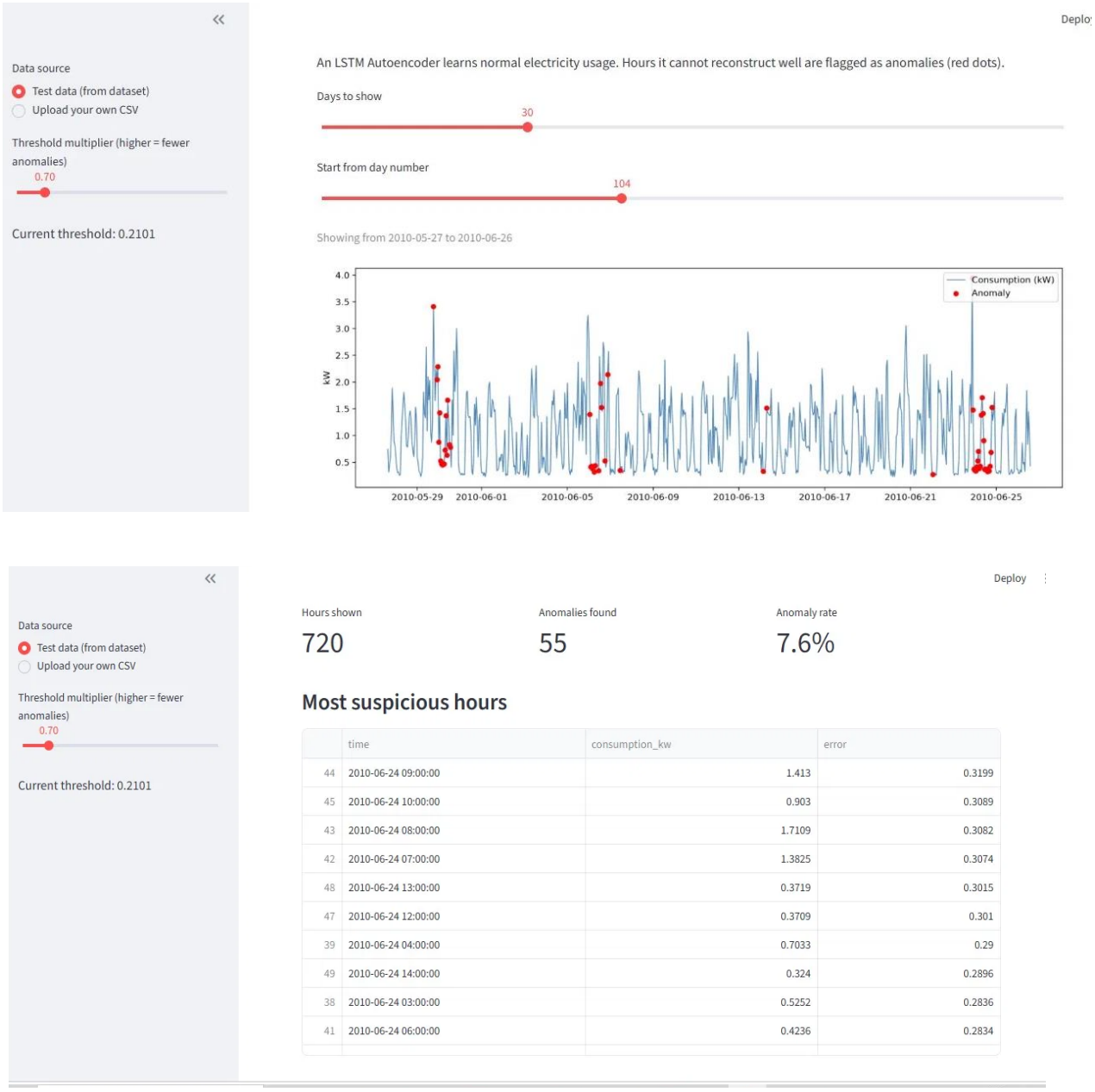


Figure 1: Dashboard with threshold multiplier 0.70 (55 anomalies)

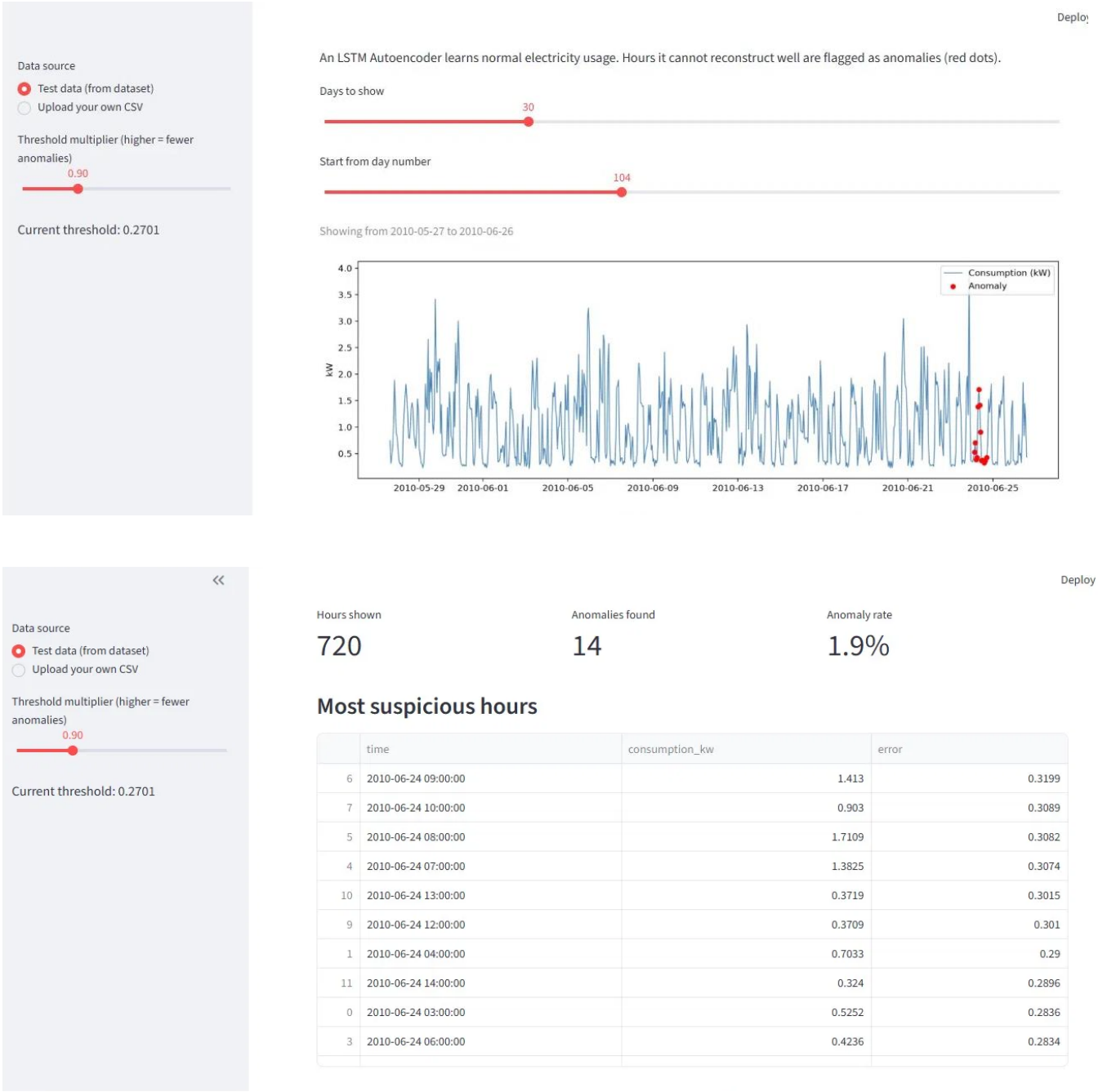


Figure 2: Dashboard with threshold multiplier 0.90 (14 anomalies)

Dashboard:

Figures 1 and 2 show the dashboard for the same 30 days (27 May to 26 June 2010) at two threshold settings. With a multiplier of 0.70 (threshold 0.2101), 55 of 720 hours (7.6%) were flagged, while with 0.90 (threshold 0.2701) only 14 hours (1.9%) were flagged. A lower threshold catches more unusual hours but may raise false alarms, while a higher threshold reports only the strongest deviations. In both cases the hours of 24 June had the highest reconstruction errors.

Discussion

The LSTM Autoencoder with maximum reconstruction error gave the best overall result (F1 = 0.640) and the highest Recall (0.512). The Z-score method had a higher Precision (0.909) but missed more anomalies (Recall 0.358), because a fixed limit cannot use the daily consumption pattern. Isolation Forest performed poorly (F1 = 0.007). The threshold controls the balance between false alarms and missed anomalies: in the dashboard, a multiplier of 0.70 flagged 55 hours (7.6%) and 0.90 flagged 14 hours (1.9%), so it is left adjustable for the user. The main limitations are that the evaluation used artificial anomalies and only one household's data.

Conclusion and Future Work

This project built an LSTM Autoencoder that detects anomalies in hourly smart meter data by learning normal 24-hour consumption patterns. On test data with injected anomalies, it achieved the best F1-score (0.640) compared with the Z-score and Isolation Forest baselines, and it is delivered as an interactive Streamlit dashboard with an adjustable threshold. In future, the system can be tested on real labelled anomalies and on data from many households, and extended with a forecasting-based approach and automatic threshold selection.

References

1. G. Hebrail and A. Berard, "Individual Household Electric Power Consumption," UCI Machine Learning Repository, 2006.
2. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
3. P. Malhotra et al., "LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection," ICML Anomaly Detection Workshop, 2016.
4. F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," *Proc. IEEE International Conference on Data Mining (ICDM)*, pp. 413–422, 2008.